DEVGUARD

DevGuard

A safety net that stops accidental Git pushes of company code to the wrong place, and shows you what happened.

Laptop agent | Control server (API) | Dashboard

PREPARED BY

Tushar Saxena

Hackathon build · October 2026

THE PROBLEM

Code leaves company laptops through ordinary mistakes, and nobody sees it happen

Wrong destination

A project is pushed to a personal, look-alike or mistyped remote.

Secrets in commits

A key, token or .env file is committed. Deleting it later does not remove it from history.

No visibility

Nobody sees who pushed what, where and when. Problems surface days later.

No proof

Exceptions are agreed informally, with no record of who approved what, or until when.

The company cannot **stop it before it happens**, cannot **see it when it does**, and has **no evidence afterwards**.

OUR ANSWER

Prevent it. Detect it. Notify. Prove it.

Prevent

Blocks a push to an unapproved destination, or one carrying prohibited files or secrets, before anything leaves.

Detect

Scans every outgoing commit, including deleted history, for secrets, keys and .env files.

Notify

Tells the developer why in plain words, and emails security administrators about blocks and secret findings.

Prove

Every push decision is an event. Every admin change goes into an append-only history.

With controlled exceptions

One project, one destination, a reason and an expiry, approved by a second admin. Developers only enroll once.

SCOPE

What DevGuard is, and is not

DevGuard is

- A safety net against honest mistakes
- Fixed, predictable rules: no guessing, no AI deciding
- Clear messages and a record of every decision

DevGuard is not

- A monitor of everything employees do
- Complete data-loss prevention
- Tamper-proof: a determined person can bypass a local hook

THE THREE PARTS

Three programs, one job each

1 · LAPTOP AGENT

Enforces

Sits in Git on each laptop and decides allow or block at push time.

Used by developers · Node.js CLI

2 · CONTROL SERVER (API)

Decides and remembers

Holds the rules, people, approvals, events and the audit history.

Used by the other two · NestJS, PostgreSQL

3 · DASHBOARD

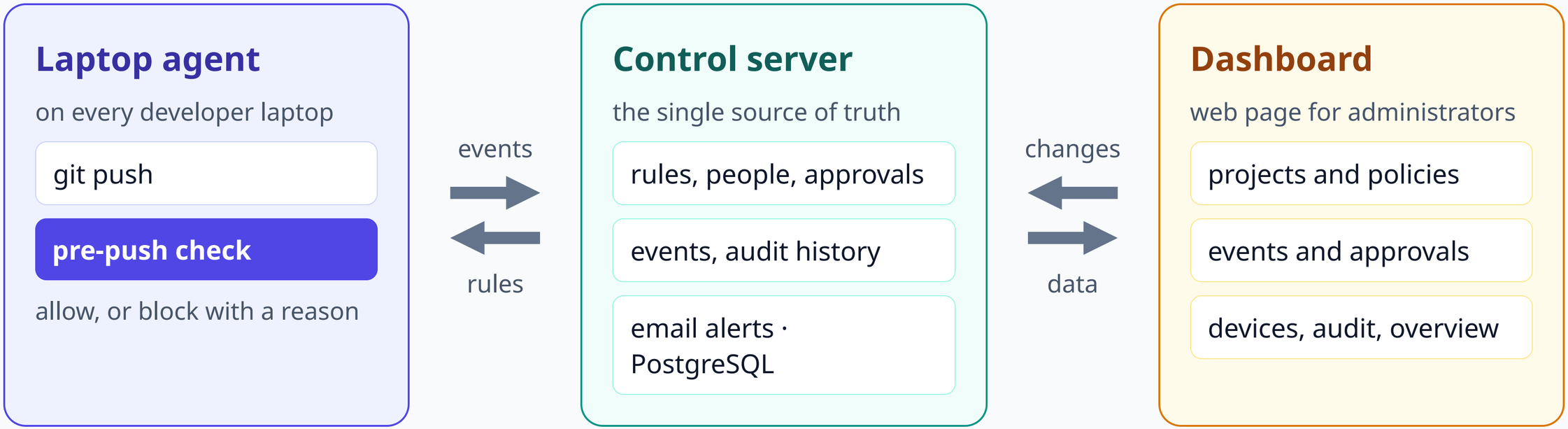
Shows and controls

Where administrators set rules and review what happened.

Used by administrators · React, Vite

HOW THEY FIT TOGETHER

Everything goes through the API



The agent and the dashboard never talk to each other. They both talk to the API.

THE FLOW

The life of one push

1 You run git push

Git runs the DevGuard hook just before sending anything.

2 Is the repo protected?

It is matched to a project assigned to this person. No match means no protection.

3 Check the destination

Compared exactly with the project’s approved destinations.

4 Check the commits

Each outgoing commit is scanned for prohibited files and known secrets.

5 Decide

Enforce blocks and explains. Audit-only allows and records.

6 Report

An event goes to the server, queued if offline. Serious ones can trigger an email.

PART 1 OF 3

The laptop agent

Its job

- Be the enforcer: allow or block each push
- Recognise which repositories belong to which project
- Keep saved rules, and queue events when offline
- Let developers ask for an exception

What it never does

- Decide policy: the server is the source of truth
- Send file contents or secret values
- Change your code or Git history
- Watch anything except Git pushes

A command-line program plus a Git hook (Node.js, TypeScript). No window or tray icon, and silent on a normal push.

WHAT A DEVELOPER SEES

A blocked push explains itself

```
$ git push fork main
DevGuard blocked this push.
  Project: Portal (confidential, enforce)
  Destination: github.com/personal/fork
  Why: the destination is not approved for this project.
  To ask for an exception:
    devguard request-exception --destination https://github.com/personal/fork --reason "<why>"
  Reference: 3f9c2e1a-…
```

The remote is untouched. The reference lets an administrator find exactly this push. Secrets are never shown, only the file, line and rule.

PART 1 OF 3

Seven commands to know

Command	What it does
devguard enroll	One-time setup: links the laptop, installs the hook
devguard status	Shows enrollment, hook, saved rules and queued events
devguard sync	Fetch the latest rules and send queued events
devguard request-exception	Ask an admin to allow a blocked destination
devguard exception status	See your requests: waiting, approved, rejected
devguard register	Link a repository to a project by hand
devguard install / uninstall	Add or remove the hook; uninstall restores Git

PART 2 OF 3

The control server (API)

Its job

- Sign-in, and a separate secret per laptop
- Projects, destinations and versioned policies
- Enrollment tokens and device revocation
- Events, exceptions and audit history
- Email alerts to administrators

Rules it enforces

- Every action is permission-checked on the server
- A second admin must approve an exception
- Scope and expiry are checked every time
- Secrets and tokens are stored only as hashes
- The audit history can only be added to

Built with NestJS, Prisma and PostgreSQL. Interactive API reference at /docs outside production.

PART 3 OF 3

The dashboard

Screen	What you do there
Overview	See blocked and allowed pushes, pending approvals, laptops
Events	Browse pushes, filter them, find one by its Reference
Approvals	Approve, reject or revoke exception requests
Projects	Set destinations, policy and who is covered
Devices	Enroll laptops, see last seen, revoke a laptop
Audit	Append-only history of every admin change

Works on phones

Light and dark theme

Keyboard and screen-reader friendly

WHAT IT SUPPORTS

Four checks on every protected push

Approved destination

Exact host and repository path. Look-alikes never match, and an address it cannot understand counts as not approved.

Address forms

HTTPS, SSH and scp-style addresses. Different spellings of one repository are treated as the same.

Prohibited files

.env files, keys and certificates, SSH private keys, .npmrc, credentials.json and more. Editable per project.

Secret patterns

Private keys, AWS, GitHub and Slack tokens and common provider tokens, including in deleted history.

WHAT IT SUPPORTS

Modes, exceptions and offline behaviour

Enforce

A problem blocks the push and explains why.

Audit-only

The push is allowed, and what would have been blocked is recorded. Good for trying a policy first.

Exceptions

One project, one destination, up to 72 hours. A second admin approves. Never waives a secret or prohibited-file block.

When the server is unreachable

Saved rules are used for up to 60 minutes. After that, enforce blocks and audit-only allows and records. Events wait in a local queue.

HOW TO USE IT

Getting started takes two short jobs

Administrator, in the dashboard

1. Create a project and add its approved destination
2. Choose the policy: Enforce or Audit only
3. Tick the people who are covered
4. Issue a one-time token, valid for 30 minutes

Developer, once on the laptop

1. Install the tool from the repository
2. Enroll with your token
3. Check it with devguard status

```
echo '<token>' | devguard enroll --server
https://devguard.example.com --token-stdin
```

After that there is nothing more to do. Rule changes arrive at the next push, or with devguard sync.

EVERYDAY USE

When a push is blocked: ask for an exception

1 · Push is blocked

The message shows the command to ask.

2 · Developer asks

devguard request-exception, with a reason.

3 · Another admin decides

Adds a reason and an expiry, up to 72 hours.

4 · Retry the push

Allowed while approved, for that destination only.

- Covers one project and one destination, for a limited time
- Never overrides a block caused by a secret or a prohibited file
- Works only while the laptop can reach the server, which checks it each time

RUNNING DEVGUARD

Quick start with Docker

```
# 1. copy the example settings
cp .env.example .env
# 2. start database, mail, API, dashboard
docker compose up -d --build
# 3. load demo users and data
pnpm --filter @devguard/api seed
```

What	Address
Dashboard	localhost:5173
API	localhost:3000
Database	localhost:5433
Mail inbox	localhost:8025

Development stack, not production. A real deployment needs HTTPS, production settings and a hardened image.

HONEST LIMITS

A safety net, not a lock

DevGuard cannot stop:

- git push --no-verify, or deleting the hook
- A repo-level core.hooksPath, such as husky
- Other Git clients and other machines
- Anything that is not a Git push
- A repository never matched to a project
- Secrets the small detector does not know

WHERE IT STANDS

Known gaps and possible next steps

Known gaps

- A compromised laptop could trigger many alert emails
- A copied session stays valid until it expires
- An ssh on port 443 exception will not match the https form
- Linux only, manual install, no auto-update

Possible next steps (not built)

- A per-laptop alert limit or digest email
- Team-lead and project-owner roles
- Notification inbox, Slack and Teams
- Packaged installer, auto-update and MDM rollout

Thank you

A safety net for honest mistakes, with a clear explanation when it steps in and a record of everything it did.

PREPARED BY

Tushar Saxena

Full documentation: DevGuard-Documentation.pdf